

CSE110A: Compilers

Topics:

- *Start of Module 4 – Optimizations*
 - *Introduction*

Module 4: Optimizations

Discussion

- What are compiler optimizations?
- Why do we want compiler optimizations?

Discussion

- What are compiler optimizations?
 - automated program transforms designed to make code more optimal
 - optimal can mean different things
 - code optimized for one system might be different for code optimized for a different system
 - we can optimize for speed, for energy efficiency, or for code size. What else?
- Why do we want the compiler to help us optimize?
 - So we can write more maintainable/portable code
 - So we don't have to worry about learning nuanced details about every possible system

Discussion

- What are some compiler optimizations you know about?

Discussion

- What are some compiler optimizations you know about?

```
for (int i = 0; i < 10; i++) {  
    x = x + 1;  
}
```

loop unrolling

```
for (int i = 0; i < 10; i++) {  
    x = x + 1;  
    i++;  
    x = x + 1;  
}
```

Discussion

- What are some compiler optimizations you know about?

```
for (int i = 0; i < 10; i++) {  
    x = x + 1;  
}
```

loop unrolling

```
for (int i = 0; i < 10; i++) {  
    x = x + 1;  
    i++;  
    x = x + 1;  
}
```

```
int foo() {  
    int i,j,k;  
    i = 10;  
    j = i;  
    k = j;  
    return k;  
}
```

constant propagation

```
int foo() {  
    int i,j,k;  
    return 10;  
}
```

Discussion

- What are some compiler optimizations you know about?

```
for (int i = 0; i < 10; i++) {  
    x = x + 1;  
}
```

loop unrolling

```
for (int i = 0; i < 10; i++) {  
    x = x + 1;  
    i++;  
    x = x + 1;  
}
```

What does this save us?

Discussion

- What are some compiler optimizations you know about?

```
for (int i = 0; i < 10; i++) {  
    x = x + 1;  
}
```

loop unrolling

```
for (int i = 0; i < 10; i++) {  
    x = x + 1;  
    i++;  
    x = x + 1;  
}
```

What does this save us?

optimizations at one stage can enable optimizations at another stage:

```
for (int i = 0; i < 10; i+=2) {  
    x = x + 2;  
}
```

provides a bigger window for local analysis

Discussion

- What are some compiler optimizations you know about?

let's do a few more

Function inlining

```
int add(int x, int y) {  
    return x + y;  
}  
  
int foo(int x, int y, int z) {  
    return add(x,y);  
}
```

```
int foo(int x, int y, int z) {  
    return x + y;  
}
```

What does this save us?
code size? speed? the ability to debug? local regions to optimize more?

Discussion

- What are some compiler optimizations you know about?

There are many more! This is an active area of research and development

For a rough metric:

`git effort` shows activities on different files and directories

clang C++/C parser: 3.5K commits

clang AST: 8.7K commits

LLVM transforms/optimizations: 30K commits

The LLVM project has been under active development since **2000**, and Clang since about **2007–2008**.

The transformation part of the code base
has the most activity by far

Discussion

- How do you enable compiler optimizations?

Discussion

- How do you enable compiler optimizations?
 - most C/C++ compilers
 - optimizing for speed
 - -O0, -O1, -O2, -O3
 - optimizing for size
 - -Os, -Oz
 - relax some constraints (especially around floating point):
 - -Ofast
 - We can use Godbolt to test various optimizations.

<https://stackoverflow.com/questions/15548023/clang-optimization-levels>

Discussion

STABILIZER: Statistically Sound Performance Evaluation

Charlie Curtsinger Emery D. Berger

Department of Computer Science
University of Massachusetts Amherst
Amherst, MA 01003
{charlie,emery}@cs.umass.edu

2013 research paper

"the performance impact of -O3 over -O2 optimizations is indistinguishable from random noise."

Discussion

- What are some of the biggest improvements you've seen from compiler optimizations?

Discussion

- What are some of the biggest improvements you've seen from compiler optimizations?
- compiler optimizations are great at well-structured, regular loops and arrays

Discussion

- What kind of transforms on your code is the compiler allowed to do?

Discussion

- What kind of transforms on your code is the compiler allowed to do?
- Matrix-add
- Many-add example
- Why did we get such a dramatic increase?

Discussion

- Extreme example

```
void foo(int * arr, int n)
{
    int i, j;
    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (arr[j] > arr[j + 1]) {
                tmp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = tmp;
            }
}
```

```
int p(int arr[], int start, int end)
{
    int pivot = arr[start];

    int count = 0;
    for (int i = start + 1; i <= end; i++) {
        if (arr[i] <= pivot)
            count++;
    }

    int pivotIndex = start + count;
    swap(arr[pivotIndex], arr[start]);

    int i = start, j = end;
    while (i < pivotIndex && j > pivotIndex) {

        while (arr[i] <= pivot) {
            i++;
        }

        while (arr[j] > pivot) {
            j--;
        }

        if (i < pivotIndex && j > pivotIndex) {
            swap(arr[i++], arr[j--]);
        }
    }

    return pivotIndex;
}

void foo(int *arr, int n)
{
    if (start >= end)
        return;

    int p = p(arr, m, n);

    foo(arr, start, p - 1);

    foo(arr, p + 1, end);
}
```

is this transform legal?

Discussion

- Extreme example

bubble sort

```
void foo(int * arr, int n)
{
    int i, j;
    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - i - 1; j++)
            if (arr[j] > arr[j + 1]) {
                tmp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = tmp;
            }
}
```

Yes this transform
would be legal!

Could any compiler figure it out?
currently unlikely..

This is a technique called
“super optimizing” and it is
getting more and more interest

```
int p(int arr[], int start, int end)
{
    int pivot = arr[start];

    int count = 0;
    for (int i = start + 1; i <= end; i++) {
        if (arr[i] <= pivot)
            count++;
    }

    int pivotIndex = start + count;
    swap(arr[pivotIndex], arr[start]);

    int i = start, j = end;
    while (i < pivotIndex && j > pivotIndex) {
        while (arr[i] <= pivot) {
            i++;
        }
        while (arr[j] > pivot) {
            j--;
        }
        if (i < pivotIndex && j > pivotIndex) {
            swap(arr[i++], arr[j--]);
        }
    }

    return pivotIndex;
}

void foo(int *arr, int n)
{
    if (start >= end)
        return;

    int p = p(arr, m, n);

    foo(arr, start, p - 1);

    foo(arr, p + 1, end);
}
```

quick sort

is this transform legal?